利用安全聚合和动态验证增强边缘计算中的联邦学习

Enhancing Federated Learning in Edge Computing With Secure Aggregation and Dynamic Verification

Mengqian Li, Youliang Tian, *Senior Member, IEEE*, Junpeng Zhang, *Member, IEEE*, Jinbo Xiong, *Senior Member, IEEE*, Xinhua Cui, and Jianfeng Ma, *Member, IEEE*

***Abstract*—Federated learning (FL) enables multiple clients to exchange gradients to facilitate collaborative training in edge computing without transferring private data to the cloud server. ~~However~~, the malicious server may disrupt the model training process by obtaining gradients to infer client information or faking validation results. ~~Considering these threats~~, we construct a dynamic, secure, and verifiable FL scheme, named DSVFL. ~~Specifically~~, we propose the fine-grained top-k gradient selection algorithm to accelerate model convergence, which can improve model accuracy by up to 49.14%. Furthermore, we propose a single-cloud secure aggregation protocol that guarantees the server cannot access the real model parameters. We also design a trusted verification algorithm to validate the server's aggregation results while resisting collusion between the server and clients. For federation dynamics, we adopt the Shamir's secret sharing in both the aggregation and verification phases, which enables the correct aggregation and decryption of ciphertexts and the independent verification of client signatures, even in cases of client dropouts. Rigorous theoretical analysis demonstrates that DSVFL can protect data privacy and ensure correct training results. Experimental results indicate that DSVFL can reduce clients' computational cost by 48%–63% and communication overhead by 33%–49% compared with existing solutions.**

***Index Terms*—Federated learning, privacy-preserving, secure aggregation, trusted verification, edge computing.**

Received 11 April 2025; revised 10 October 2025; accepted 16 October 2025. Date of publication 27 October 2025; date of current version 6 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62272123 and Grant 62272102, in part by the Science and Technology Program of Guizhou Province under Grant [2020]5017 and Grant [2022]065, in part by the Natural Science Foundation of Fujian Province under Grant 2023J02014, in part by the S&T Program of Shijiazhuang under Grant 2517902907A, and in part by the S&T Program of Hebei Province under Grant F2025205017 and Grant 25240701D. Recommended for acceptance by C. H. Liu. *(Corresponding authors: Youliang Tian; Jinbo Xiong.)*

Mengqian Li is with the State Key Laboratory of Public Big Data, College of Computer Science and Technology, Guizhou University, Guiyang 550025, China, and also with the Hebei Provincial Key Laboratory of Network and Information Security, Hebei Normal University, Shijiazhuang 050024, China (e-mail: mqli88@163.com).

Xinhua Cui is with the State Key Laboratory of Public Big Data, College of Computer Science and Technology, Guizhou University, Guiyang 550025, China (e-mail: gs.xhcui21@gzu.edu.cn).

Youliang Tian is with the State Key Laboratory of Public Big Data, College of Big Data and Information Engineering, Guizhou University, Guiyang 550025, China (e-mail: youliangtian@163.com).

Junpeng Zhang is with the Hebei Provincial Key Laboratory of Network and Information Security, Hebei Normal University, Shijiazhuang 050024, China (e-mail: zhangjunpeng@hebtu.edu.cn).

Jinbo Xiong is with the Industrial School of Joint Innovation, Quanzhou Vocational and Technical University, Quanzhou 362268, China, and also with the Fujian Provincial Key Laboratory of Network Security and Cryptology, College of Computer and Cyber Security, Fujian Normal University, Fuzhou 350117, China (e-mail: jbxiong@qvtu.edu.cn).

Jianfeng Ma is with the State Key Laboratory of Integrated Services Network, School of Cyber Engineering, Xidian University, Xi'an 710071, China (e-mail: jfma@mail.xidian.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3625654

I. Introduction

THE seamless integration of emerging industries such as artificial intelligence, the Internet of Things, and cloud computing has rendered data an invaluable resource. Consequently, this has promoted more stringent protection requirements for data sharing. Federated learning (FL) [1], [2] is characterized by the training of models through transmitting model parameters without exchanging the dataset, providing a new approach to data protection in edge computing. Inherently resolves the predicament of data silos across diverse application scenarios [3], [4], [5], thereby facilitating research on decentralized data. However, FL still faces communication and security bottleneck issues that constrain its development.

The rapid expansion of deep neural networks in the communication domain of edge computing, driven by the dramatic increase in data size, has been remarkable. Unfortunately, this has also brought considerable model parameter exchanges during iterative training, resulting in substantial communication overhead [6], [7]. Recent study [8] indicates that the airtime required for a single training round in LoRaWAN-based FL systems reaches up to 52.8 minutes in industrial IoT scenarios, highlighting the significant communication burden in resource-constrained environments. In addition, in practical FL applications, most intermediate parameters generated through multi-party interactions are typically encrypted [9], [10], further amplifying the communication overhead. From another perspective, communication limitations also pose a significant challenge. During training, clients maintain absolute control over their devices and can cease communication with the server at any time. Client devices frequently disconnect due to network speed or bandwidth issues. The above situations form federation dynamics, which diminishes the training efficiency of the model.

FL also confronts a series of security threats [11], including the danger of data privacy disclosure [12] and model destruction [13]. Although there is no need to share the client's original dataset, numerous methods exist to infer information about the training data. Particularly, when the server acts as a malicious attacker, it can analyze the uploaded model parameters to infer client privacy. Moreover, the server may conspire with some clients to tamper with the validation results when some researchers design the verification algorithm to validate the server aggregation results. For example, in industrial quality inspection

1536-1233 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.

1. Sattler 2020（STC）
上行做完稀疏 → 直接量化成三值（-μ, 0, +μ）→ Golomb 编码 → 上传→
目标：把通信压到最小，纯效率。
2. Li 2026（DSVFL-Ftop-k）
上行做完稀疏 → 不量化、不编码 → 直接加密（Paillier）→ 安全聚合 → 抗合谋验证→
目标：安全 + 隐私 + 可验证 + 动态掉线兼容

scenarios, malicious servers may tamper with the accuracy of federated models, fabricating overly optimistic results and altering corresponding validation outcomes. This could mislead manufacturers into deploying unreliable systems, creating serious security risks and economic losses.

To address communication and security issues in FL, numerous schemes for gradient compression [14], [15], [16] and privacy protection [17], [18] have been proposed in academia. Nevertheless, existing schemes often contemplate reducing the communication overhead and increasing the strength of privacy protection at the expense of model accuracy, which cannot train high-quality models and are difficult to apply in practice. We conduct research focusing on the deficiencies in existing schemes and propose a dynamic, secure, and verifiable FL scheme (DSVFL). DSVFL is designed to resist malicious server inference and information tampering while ensuring efficient computational and communication in FL. The specific contributions of DSVFL are outlined as follows.

- We propose the fine-grained top-k gradient selection algorithm (Ftop-k) to accelerate model convergence and improve the quality of model training. This algorithm can significantly improve model accuracy by up to 49.14%.
- We construct a lightweight data secure aggregation protocol (SCSA) for single-cloud model, enabling secure aggregation of local training data during global model training. The SCSA is designed based on the threshold Paillier technique, which makes the local gradient computationally invisible to the rest of clients and the aggregation server.
- We design a trusted verification algorithm to validate the server aggregation results, which can resist the collusion verification between the server and clients. Moreover, we improve the shamir's secret sharing technique and apply it to the client aggregation and verification phases to eliminate the impact of federation dynamics on model training.
- We analyze the correctness and security of DSVFL and conduct experiments to evaluate its performance across three network models of varying complexity. The results demonstrate that the scheme effectively protects the owner's data and is suitable for dynamic federation environments, ensuring efficient aggregation verification. Moreover, our scheme significantly reduces computational cost by 48%-63% and communication overhead by 33%-49%.

The remainder of the paper is organized as follows. Section II presents the system overview, threat model, design goals and preliminaries. Section III depicts a series of algorithms of the DSVFL. Section IV provides the theoretical analysis of DSVFL. Section V demonstrates the experimental results and the related work is reviewed in Section VI. Finally, we conclude this paper in Section VII.

II. Problem Statement and Preliminaries

This section provides an elaborate explanation of the system overview, threat model, design goals and outlines the fundamental concept essential for designing the DSVFL scheme.

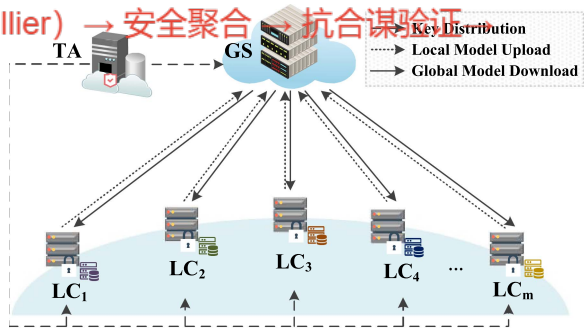


Fig. 1. The system architecture of DSVFL.

A. System Overview

We mainly concentrate on designing a FL system that could satisfy the requirement of dynamic and secure verifiable, as shown in Fig. 1. The DSVFL system comprises three entity types: trusted authority (TA), global server (GS), and local clients (LCs).

- *TA:* The TA is considered a trusted third-party authority responsible for system initialization, key distribution, and trusted verification, but not participating in FL training.
- *GS:* The GS is a malicious service provider mainly charged with collecting and aggregating data, and it may infer or tamper with the collected data.
- *LCs:* LCs join because they need to obtain the global model that trains with other local data. We assume that some LCs may collude with others and the GS to disrupt training.

B. Threat Model

In the DSVFL, the threat model is specified as follows. The TA is considered trustworthy and will not collude with LCs and GS. LCs may act as *honest-but-curious* entities. Throughout the entire network system, some LCs attempt to infer others' private information from the obtained ciphertext, while still honestly contributing local encrypted data. Specifically, we allow GS to collude with some LCs to obtain the most aggressive capability, such as the LC sharing its local intermediate information with the GS without violating the agreement process, thereby assisting the GS in launching attacks.

The GS is considered malicious. It is assumed to operate as a probabilistic polynomial-time adversary. Specifically, during the FL process, the GS gains access to all ciphertext data within the system. Driven by economic interests, the GS attempts to recover plaintext data from the ciphertext submitted by LCs. Furthermore, it may disregard the protocol and collude with LCs to arbitrarily alter aggregation outcomes and deceive devices into accepting misleading aggregation results, ultimately skewing the resulting training model to its advantage.

C. Design Goals

The main objective of our investigation is to design a verifiable and efficient dynamic FL protection scheme that addresses the privacy threats and the overhead problem currently confronting multi-party collaborative training. This solution achieves secure data aggregation, trustworthy GS authentication, dynamic adjustment of training states, and efficient accuracy in training models.

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.

- *Confidentiality:* The primary objective of our proposed scheme, DSVFL, is to guarantee data privacy. During FL model updates, the LC's local training data must not be disclosed to the GS or other LCs, thereby protecting influence from inference attacks.
- *Verifiability:* The DSVFL scheme should guarantee verifiability in dynamic FL. This prevents the GS and external adversaries from tampering with the data during the training process to affect the model training.
- *Dynamics:* The DSVFL scheme should be suitable for federation dynamic environments, eliminating the impact of LC dropouts on model aggregation and trusted verification.
- *Fidelity:* The DSVFL scheme should provide privacy protection while maintaining high accuracy. Our scheme does not sacrifice accuracy for privacy protection and achieves high-accuracy model training while protecting sensitive data.
- *Efficiency:* The DSVFL scheme should exhibit high computational and communication efficiency. While ensuring that the correct model training operations are performed at the GS and LCs, our scheme reduces system computational cost and communication overhead, thereby accelerating training speed.

D. Paillier Encryption

Paillier encryption [19] is an additively homomorphic scheme designed to perform operations on encrypted data. When the encrypted data is decrypted, the result is equivalent to performing the operations directly on the plaintext data.

- *Key Generation:* Given the security parameter ρ, pick two large prime numbers p and q of equal length and ρ satisfying $gcd[p \cdot q, (p-1)(q-1)] = 1$. Compute $n = p \cdot q$ and $\lambda = lcm(p-1, q-1)$, where lcm denotes the least common multiple. Choose a random number g such as $g \in \mathbb{Z}^*_{n^2}$. Define the function $\mathcal{J}(x) = \frac{x-1}{n}$ and calculate $\mu = [\mathcal{J}(g^\lambda \bmod (n^2))]^{-1} \bmod (n)$. Then, the public key is output as $pk = (n, g)$ and the private key is $sk = (\lambda, \mu)$.
- *Encryption:* Input a plaintext message W that satisfies $W \in \mathbb{Z}_n$. Choose the random number $\gamma \in \mathbb{Z}^*_{n^2}$ and compute the ciphertext as

$$E[[W]] = g^W \gamma^n \bmod \left(n^2\right). \tag{1}$$

- *Decryption:* Input the ciphertext message $E[[W]] \in \mathbb{Z}^*_{n^2}$, compute the plaintext as

$$W = \left[\mathcal{J}\left(W^\lambda \bmod \left(n^2\right)\right)\right] \cdot \mu \bmod (n). \tag{2}$$

- *Additive Homomorphism Property:* For any plaintext $W_1, W_2 \in \mathbb{Z}_n$ and any $\gamma_1, \gamma_2 \in \mathbb{Z}^*_n$, the corresponding ciphertexts are

$$\begin{aligned} E[[W_1]] \cdot E[[W_2]] &= g^{W_1+W_2} \cdot (\gamma_1+\gamma_2)^n \bmod n^2 \\ &= E[[W_1+W_2]], \end{aligned} \tag{3}$$

i.e., multiplication of ciphertext is equal to the result of encryption after addition of plaintext.

TABLE I
THE DESCRIPTION OF MAIN SYMBOLS

Symbols	Description
$\nabla\hat{\omega}_i^t$	Gradients of the LC$_i$ in the t-th round.
$\nabla\omega_i^t$	Gradients after error-compensated of the LC$_i$ in the t-th round.
Y_i^t	The error buffer for compensation in the t-th round.
d	The number of gradients during model training.
n	The product of two large prime numbers p and q.
n_i	The i-th shamir's secret sharing of the public key modulus n.
$E[\![W_i]\!]$	The encrypted data of the LC$_i$.
$E[\![W]\!]$	The aggregated encryption data of the GS.
W	The decryption data of the GS.
Z	The random number used for the signature.
z_{ij}	LC$_i$'s j-th share after secret sharing of Z.
σ_i	The hash value of the ciphertext $E[\![W_i]\!]$.
Z_{ij}	The j-th signature generated by LC$_i$ for distribution to LC$_j$.
S_j	The signature received by LC$_j$.
ϖ_i	The aggregated signature of LC$_i$.
R_i	The validation result of LC$_i$.

III. CONSTRUCTION OF DSVFL SCHEME

In this section, we concretely sketch the DSVFL scheme. The scheme contains three main parts: Ftop-k algorithm, SCSA protocol, and trusted verification. Moreover, the key symbols and their description are summarized in Table I.

A. Fine-Grained Top-k Gradient Selection

The DSVFL system comprises m LCs, each of which is trained locally on its private dataset. The trained local model gradients are then uploaded to the GS for aggregation. During model gradient uploading, the number of gradient transmissions should be reduced by model compression to minimize communication overheads while maintaining model accuracy. To this end, we improve the traditional top-k algorithm and design the Ftop-k scheme, as shown in Algorithm 1. We introduce an accumulable residual buffer in Ftop-k to store the gradient information not selected by top-k, and superimpose it with the current gradient to form a compensated gradient in the next round. This approach does not rely on reference point updating or momentum structure, and achieves an organic combination of error compensation and sparse communication. Ftop-k can improve model training quality with the same communication overhead as the top-k. Our optimization objective is to solve the following problem:

$$\min \sum_{i=1}^{m} \mathcal{C}\left(\nabla\hat{\omega}_i^t, r\right),$$

$$\text{s.t.} \frac{1}{m}\sum_{i=1}^{m} L_{top-k}\left(\omega_i^t, D\right) \geq \frac{1}{m}\sum_{i=1}^{m} L_{Ftop-k}\left(\omega_i^t, D\right),$$

where $\mathcal{C}(\cdot)$ represents the communication overhead, r is the gradient compression rate, $L(\cdot)$ denotes the loss function for model training, ω_i^t represents the model weights of the LC$_i$ of the round t. The implementation steps are as follows.

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.

1) Initialization: In the initial stage of DSVFL system training, each LC generates an error buffer for storing the gradient error Y_i. The gradient errors in the error buffer are all set to 0 before the first epoch of training starts.

2) Gradients Calculation: After local training for round t, LCs calculate the model gradients, which is defined as follows

$$\nabla\hat{\omega}_i^t = \frac{\omega_i^t - \omega^t}{\eta}, \tag{4}$$

where η is the learning rate and ω^t is the global weights at round t.

Gradients $\nabla\hat{\omega}_i$ are then error compensated using the gradient error stored in the error buffer to obtain the compensated gradients.

$$\nabla\omega_i^t = \nabla\hat{\omega}_i^t + Y_i^t. \tag{5}$$

3) Ftop-k Gradients Selection: LCs perform gradient compression based on the compensated gradients $\nabla\omega_i^t$ before uploading gradients to GS. Specifically, the top k of these gradients are selected for transmission, and we obtain

$$\text{Ftop}{-}k\left(\nabla\omega_i^t\right) = \arg\max_{|S|=k} \left|\nabla\omega_i^t\left[j\right]\right|, j \in S, \tag{6}$$

where S denotes the set of selected gradient indices and Ftop-$k(\nabla\omega_i^t)$ are selected gradients.

4) Gradient error updating: After selecting k gradients, LCs calculate and update the error buffer for compensation at the next round as follows

$$Y_i^{t+1} = \nabla\omega_i^t - \text{Ftop}{-}k\left(\nabla\omega_i^t\right). \tag{7}$$

In this step, LCs update the error buffer every round based on the latest local model and compute the top k gradient for uploading based on this buffer. Unlike traditional top-k gradient compression, we accumulate unselected gradients in the error buffer, which can reduce the information loss from gradient compression and improve model accuracy.

In the DSVFL system, we introduce a Ftop-k algorithm with an error compensation mechanism. Each LC independently maintains its own error buffer, which can more accurately compensate for the bias caused by local data distribution while reducing the communication overhead. Suppose the number of gradients during model training is d, and we analyze the algorithmic complexity of applying the Ftop-k algorithm for dataset size U. The computational complexity of computing gradients for LCs at each round is denoted as $O(Ud)$. The gradients are then compensated for errors with an $O(d)$ computational complexity. The k gradients are selected using a heap sort or fast selection algorithm with an $O(d\log k)$ computational complexity. Finally, the computational complexity of gradient error update is $O(d)$. Since U is a large number in practice, $O(d)$ is negligible compared to $O(Ud)$. The final computational complexity of the Ftop-k algorithm after simplification is $O(Ud + d\log k)$, the computational complexity of the top-k algorithm is also $O(Ud + d\log k)$, and the computational complexity of the random-k algorithm is $O(Ud)$, which we have shown in Table II. The above comparison shows that the Ftop-k algorithm, with the same amount of communication as the top-k algorithm, ultimately leads to better model convergence.

TABLE II
COMPLEXITY COMPARISON

Scheme	Communication Complexity	Computation Complexity
top-k [20]	$O(k)$	$O(Ud + d\log k)$
random-k [21]	$O(k)$	$O(Ud)$
Ftop-k	$O(k)$	$O(Ud + d\log k)$

算法一：细粒度Top-k梯度选择。

Algorithm 1: Fine-Grained Top-k Gradient Selection.

Input: Global model ω, local model ω_i .
Output: Ftop-k gradients Ftop-k $(\nabla\omega_i)$.

▷ **LC**$_i$ **do** ◁
Initialize $Y_i^0 = [0, \cdots, 0]$
for each global round t from 1 to T **do**
  ▷ *calculate gradients* ◁
  $\nabla\hat{\omega}_i^t = \frac{\omega_i^t - \omega^t}{\eta}$
  $\nabla\omega_i^t = \nabla\hat{\omega}_i^t + Y_i^t$
  ▷ *select Ftop-k gradients* ◁
  Ftop-k $(\nabla\omega_i^t) = \arg\max_{|S|=k} |\nabla\omega_i^t[j]|, j \in S$
  ▷ *update gradient error* ◁
  $Y_i^{t+1} = \nabla\omega_i^t -$ Ftop-k $(\nabla\omega_i^t)$
return Ftop-k $(\nabla\omega_i^t)$

B. SCSA: Single-Cloud Secure Aggregation Protocol

We improve the Paillier encryption scheme based on the Shamir's secret sharing technique to allow for LCs dropout. The SCSA protocol remains resilient to the malicious GS, which achieves single-cloud secure aggregation and accurate decryption of LCs. Algorithm 2 shows the detailed.

1) Initialization: TA produce the private key $sk_\rho = (\mu, \lambda)$ and the public key $pk_\rho = (n, g)$ with the security parameter ρ, where $ord(g) = kn$, n is an element in the finite field $GF(\hat{p})$ and $\hat{p}$ is a large prime number satisfying $n < \hat{p}$. In particular, longer security parameters improve security, but the encryption and decryption time rise significantly, and the communication overhead also grows linearly with the length of the security parameter. Subsequently, TA randomly selects a large integer α that is coprime with ρ, satisfying $|\alpha| < \frac{\rho}{2}$ and $\gcd(k, \alpha) = 1$. They compute $h = g^\alpha \bmod n^2$ and publicly disclose the system parameters $PP = \langle\rho, n, g, h\rangle$.

To generate the private keys for LCs, TA first picks a random $(\tau-1)$-th degree polynomial: $f(x) = a_0 + a_1 x + \cdots + a_{\tau-1}x^{\tau-1}$, where $a_0 = n$. The setting of τ does not directly affect the FL model training accuracy. However, it affects the time of model updates obtained during the aggregation process. When τ is set to a large value, more LCs must collaborate to complete secret recovery, significantly enhancing the security of the FL system. This approach also reduces the tolerance for LC dropouts. Conversely, when τ is set to a small value, the training can continue to complete smoothly even when LCs drop out,

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.

算法2：SCSA：单云安全聚合。

Algorithm 2: SCSA: Single-Cloud Secure Aggregation.

Input: Security parameter ρ, private key $sk_\rho = (\mu, \lambda)$, public key $pk_\rho = (n, g)$, large integer α, random number θ_t, plaintext data W_i.
Output: Aggregation plaintext W.

▷ TA do ◁
Generate a random $(\tau-1)$-th degree polynomial: $f(x) = a_0 + a_1 x + \cdots + a_{\tau-1}x^{\tau-1}$
$n_i = f(x_i)$
$h = g^\alpha \bmod n^2$, $\gamma_i = \theta_t^{n_i} \bmod n^2$
$sk_{LC_i} = \langle \gamma_i, \mu, \lambda, \alpha \rangle$
▷ LC_i do ◁
$W_i = b \cdot \nabla\omega_i$
$E[[W_i]] = g^{W_i} \cdot h^{\beta_i} \cdot \theta_t^{n_i} \bmod n^2$
▷ GS do ◁
$E[[W]] = \prod_{i=1}^{m} E[[W_i]] \bmod n^2$
▷ LC_i do ◁
$W = ((\mathcal{J}(E[[W]])^\lambda \bmod n^2) \cdot \mu \bmod n) \bmod \alpha$
return W

offering higher fault tolerance and relatively lower communication overhead. But an excessively low threshold compromises system security. Then, the public key n of Paillier's encryption algorithm is divided into m random numbers $n_1, n_2, \ldots, n_m$ by Shamir's secret sharing. TA chooses a random number $\theta_t \in \mathbb{Z}_n^*$ as the task ID for the t-th round of training and computes a partial private key $\gamma_i = \theta_t^{n_i} \bmod n^2$ for each LC. Finally, TA transmits $sk_{LC_i} = \langle \gamma_i, \mu, \lambda, \alpha \rangle$ as a private key to the corresponding LC.

2) Encryption: Given the LC_i plaintext data $W_i = b \cdot \nabla\omega_i$, where b is a large integer capable of converting model parameters to integers, W_i is encrypted with a private key for convenience. The encryption operation is as follows

$$E[[W_i]] = g^{W_i} \cdot h^{\beta_i} \cdot \theta_t^{n_i} \bmod n^2, \tag{8}$$

where β_i is a random number satisfying $|\beta_i| < \frac{\rho}{2}$.

3) Aggregation: The m LCs upload their respective ciphertext data $E[[W_i]]$, and the GS performs the aggregation operation represented as

$$E[[W]] = \prod_{i=1}^{m} E[[W_i]] \bmod n^2. \tag{9}$$

4) Decryption: Once GS aggregates the ciphertexts uploaded from all LCs, it sends the aggregation results to LCs for decryption. LCs decrypt using the private key, and the decryption process follows

$$W = \left(\left(\mathcal{J}(E[[W]])^\lambda \bmod n^2 \right) \cdot \mu \bmod n \right) \bmod \alpha. \tag{10}$$

C. Trusted Verification

To verify whether the malicious GS performs secure aggregation, we design a trusted verification, as described in Algorithm 3. Additionally, the algorithm can resist collusive verification of $\tau - 1$ LCs with the malicious GS. In the real-world scenario, it is both costly and challenging for the GS to conspire with multiple LCs, which ensures the algorithm's effectiveness.

算法3：可信验证

Algorithm 3: Trusted Verification.

Input: The number of LCs m, the random $(\tau-1)$-th degree polynomial: $o_i(x) = c_{i0} + c_{i1}x + \cdots + c_{it-1}x^{t-1}$, ciphertext data $E[\![W_i]\!]$.
Output: Verification result R_e.

▷ TA do ◁
Generate cyclic group $\mathbb{G}$ of prime order q with generator g
$H_1 : \{0,1\}^* \to \mathbb{G}$
$Z = Rand(\mathcal{X})$ and send to all LCs
Generate m polynomial: $o_i(x), 1 \le i \le m$
Divide Z into m parts $z_{ij} = o_i(x_j), 1 \le j \le m$
Distribute $z_i = \{z_{1i}, z_{2i}, \cdots, z_{mi}\}$ to $LC_i, 1 \le i \le m$
Random select a LC_e
▷ LC_e do ◁
$\sigma_e = H_1(E[\![W_e]\!])$
Send e and σ_e to other LCs and TA
▷ LC_j do ◁
Receive e and σ_e
Compute $Z_{ej} = \sigma_e^{z_{ej}}$
▷ GS do ◁
$\varpi_e = \prod_{j=1}^{m} Z_{ej}$
Send ϖ_e to TA
▷ TA do ◁
if $\varpi_e = \sigma_e^Z$ **then**
  $R_e = True$
  Distribute $E[\![W]\!]$ to all LCs
else
  $R_e = False$
  Terminate training
return R_e

1) Initialization: The TA generates cyclic group $\mathbb{G}$ of prime order q with generator g. A secure hash function is defined as $H_1 : \{0,1\}^* \to \mathbb{G}$. The TA generates a random number Z based on the random seed $\mathcal{X}$, where Z is an element in the finite field $GF(\hat{q})$, and $\hat{q}$ is a large prime number satisfying $Z < \hat{q}$.

TA generates m polynomial: $o_i(x), 1 \le i \le m$, and divides Z into m parts according to the polynomial $o_i(x_j) = c_{i0} + c_{i1}x_j + \cdots + c_{i\tau-1}x_j^{\tau-1}$ to obtain $\{z_{i1}, z_{i2}, \ldots, z_{ij}, \ldots, z_{im}\}$,

$$\begin{cases} z_{i1} = o_i(x_1) = c_{i0} + c_{i1}x_1 + \cdots + c_{i\tau-1}x_1^{\tau-1} \\ z_{i2} = o_i(x_2) = c_{i0} + c_{i1}x_2 + \cdots + c_{i\tau-1}x_2^{\tau-1} \\ \cdots \\ z_{im} = o_i(x_m) = c_{i0} + c_{i1}x_m + \cdots + c_{i\tau-1}x_m^{\tau-1}. \end{cases} \tag{11}$$

Then, TA classifies the secret sharing to obtain $z_j = \{z_{1j}, z_{2j}, \ldots, z_{mj}\}$ and distributes z_j to the corresponding LC_j. Subsequently, LC_e is randomly selected as the verification LC.

2) Signature Calculation: Subsequently, LC_e computes the hash value of the ciphertext $\sigma_e = H_1(E[[W_e]])$ and then transmitted it along with the identifier e to all LCs. LC_j receives and computes $Z_{ej} = \sigma_e^{z_{ej}}$, where $1 \le j \le m$ and z_{ej} represents the

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.

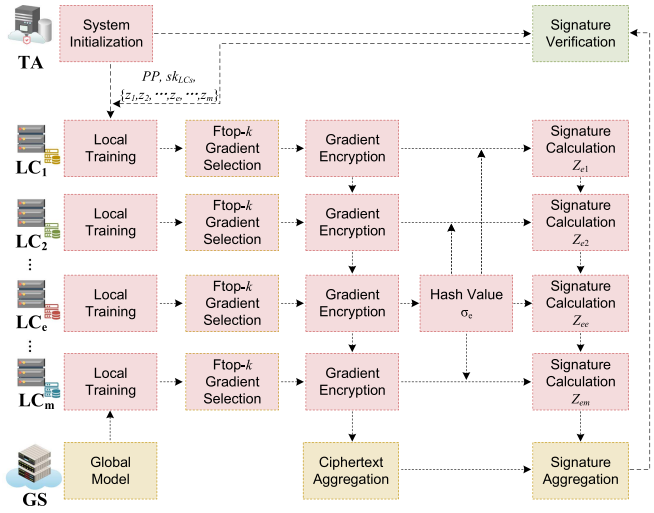


Fig. 2. The overview of DSVFL.

e-th secret sharing sent by TA to LC_j. After that, signatures Z_{ej} and encrypted data are uploaded to GS.

3) Signature Aggregation: The GS receives signatures Z_{ej} and encrypted data sent by LCs and products aggregation operation, where the signature aggregation is represented as follows

$$\varpi_e = \prod_{j=1}^{m} Z_{ej}. \tag{12}$$

Then, the GS returns the aggregated encryption result with the aggregated signature ϖ_e to the TA.

4) Signature Verification: TA performs correctness verification on the aggregated signature. If $\varpi_e = \sigma_e^Z$ is satisfied, the GS is judged to have performed the aggregation operation correctly, and the validation result R_e returns True; if this equation is not valid, the validation result returns False, and the training is terminated.

$$R_e = \begin{cases} True & \text{if } \varpi_e = \sigma_e^Z \\ False & \text{if } \varpi_e \neq \sigma_e^Z. \end{cases} \tag{13}$$

D. Description of DSVFL

In this subsection, we provide a detailed description of the overall DSVFL scheme and Fig. 2 is the overview of the DSVFL.

Specifically, the DSVFL encompasses four distinct stages: initialization, local data encryption, secure aggregation, and data verification and decryption. This scheme can resist collusion verification between the GS and fewer than $\tau - 1$ LCs.

1) Initialization: First, the TA executes the key generation process and distributes private keys $sk_{LC_i} = \langle \gamma_i, \mu, \lambda, \alpha \rangle (1 \leq i \leq m)$ to LCs, facilitating data encryption and decryption. Then, the TA generates a cyclic group $\mathbb{G}$ of prime order q with generator g, configures a secure hash function, and generates a random number Z as well as corresponding secret sharing $z_j (1 \leq j \leq m)$ to LCs to enable verification of the GS aggregation results. Besides, LCs receive the global model initialized by the GS.

2) Local Data Encryption: Assuming each LC receives the global model from the GS and computes the corresponding local model gradients on local dataset D_i as follows

$$\nabla \omega_i = \nabla L(\omega_i, D_i). \tag{14}$$

Furthermore, LCs utilize the private key issued by the TA to encrypt plaintext data derived from Ftop-k algorithm. Before encryption, $\nabla \omega_i$ must be converted to an integer W_i by multiplying it with a large integer b, typically chosen as 1000 or larger to enhance precision.

Concurrently, LC_e calculates the hash value σ_e and shares it with the identifier e to other LCs. Specifically, LC_i generates the signature using secret sharing. Then, LC_i uploads the ciphertext data $E[[W_i]]$ to GS, as well as the signature.

3) Secure Aggregation: Upon receiving encrypted data and signatures sent by LCs, GS aggregates them separately to derive the signature aggregation result ϖ_e of all LCs, the ciphertext aggregation result $E[[W]]$. GS is not endowed with the decryption capability in this aggregation operation and is restricted to performing only the aggregation. Following this, the GS returns the aggregation signature ϖ_e along with the aggregation result to the corresponding TA.

4) Data Verification and Decryption: TA verifies the aggregated results against forgery using aggregated signatures. If valid, TA distributes the ciphertext aggregation result to all LCs for decryption. Otherwise, the training is terminated.

IV. Theoretical Analysis

This section presents an analysis of the correctness, security, and collusion resistance of the DSVFL.

A. Correctness Analysis

Theorem 1: The SCSA algorithm within the proposed DSVFL scheme permits the dropout of up to $m - \tau$ LCs, while the GS can still achieve secure aggregation. LCs receive the aggregated ciphertext to enable correct decryption and get the correct aggregation result.

Proof: GS starts the aggregation process after receiving the ciphertext data uploaded by LCs as

$$\begin{aligned} E[[W]] &= \prod_{i=1}^{m} E[[W_i]] \bmod n^2 \\ &= \prod_{i=1}^{m} g^{W_i} \cdot h^{\beta_i} \cdot \theta_t^{n_i} \bmod n^2 \\ &= g^{\sum_{i=1}^{m} W_i} \cdot h^{\sum_{i=1}^{m} \beta_i} \cdot \prod_{i=1}^{m} \theta_t^{n_i} \bmod n^2 \\ &= g^{\sum_{i=1}^{m} W_i + \alpha \sum_{i=1}^{m} \beta_i} \cdot \theta_t^{n} \bmod n^2. \end{aligned} \tag{15}$$

From (15), it can be seen that after aggregation, it is possible to obtain a valid ciphertext for Paillier homomorphic encryption $E[[W]]$. LCs can decrypt the received aggregated ciphertext and achieve correct decryption as follows

$$\begin{aligned} W &= \left(\left(\mathcal{J}(E[[W]])^{\lambda} \bmod n^2 \right) \cdot \mu \bmod n \right) \bmod \alpha \\ &= \left(\sum_{i=1}^{m} W_i + \alpha \sum_{i=1}^{m} \beta_i \right) \bmod \alpha. \end{aligned} \tag{16}$$

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.

Therefore, the above proof ensures the correct aggregation and decryption of LCs sensitive data. Next, we discuss LCs' dynamic dropout situations. If $m-\tau$ LCs drop out, i.e., τ LCs upload ciphertext data, GS can still complete secure aggregation as

$$\begin{aligned} & E\left[[W]\right] \\ &= \prod_{i=1}^{\tau} E\left[[W_i]\right] \bmod n^2 \\ &= \prod_{i=1}^{\tau} g^{W_i} \cdot h^{\beta_i} \cdot \theta_t^{n_i} \bmod n^2 \\ &= g^{\sum_{i=1}^{\tau} W_i} \cdot h^{\sum_{i=1}^{\tau} \beta_i} \cdot \prod_{i=1}^{\tau} \theta_t^{n_i} \bmod n^2 \\ &= g^{\sum_{i=1}^{\tau} W_i + \alpha \sum_{i=1}^{\tau} \beta_i} \cdot \theta_t^{\sum_{i=1}^{\tau} q(x_i) \prod_{i=1, j \neq i}^{\tau} \frac{-x_j}{x_i - x_j}} \bmod n^2 \\ &= g^{\sum_{i=1}^{\tau} W_i + \alpha \sum_{i=1}^{\tau} \beta_i} \cdot \theta_t^{q(0)} \bmod n^2 \\ &= g^{\sum_{i=1}^{\tau} W_i + \alpha \sum_{i=1}^{\tau} \beta_i} \cdot \theta_t^{n} \bmod n^2. \end{aligned} \tag{17}$$

■

In summary, the SCSA algorithm can tolerate a certain number of dropped LCs and is dynamically fault-tolerant.

Theorem 2: The trusted verification algorithm in the DSVFL scheme allows fewer than or equal to $m-\tau$ LCs to drop out. The GS guarantees that the signature aggregation can still be completed, and LCs can still correctly verify the aggregated signature from the GS.

Proof: When there are $m-\tau$ LCs dropped, GS receives signatures uploaded by τ LCs and performs signature aggregation as:

$$\begin{aligned} \varpi_e &= \prod_{j=1}^{\tau} Z_{ej} = \prod_{j=1}^{\tau} \sigma_e^{z_{ej}} = \sigma_e^{\sum_{j=1}^{\tau} z_{ej}} \\ &= \sigma_e^{\sum_{j=1}^{\tau} o_e(x_j) \prod_{l=1, j \neq l}^{\tau} \frac{-x_l}{x_j - x_l} \bmod q} = \sigma_e^{o_e(0)} = \sigma_e^{Z}. \end{aligned} \tag{18}$$

■

It follows that GS can tolerate a certain number of LCs dropout when aggregating signatures. Trusted verification can still be completed when at least τ LCs participate, verifying whether $\varpi_e = \sigma_e^Z$ holds.

B. Security Analysis

Theorem 3: LCs encrypt the plaintext data by (8) to generate the ciphertext data $E[[W_i]]$ at the end of local training. Therefore, the ciphertext data $E[[W_i]]$ is semantically secure under the chosen plaintext attack (IND-CPA).

Proof: Assuming the existence of a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ and a challenger C, a game between them is used to prove the semantic security of the ciphertext data $E[[W_i]]$.

1) Initialization: The challenger C generates the private key $sk_\rho = (\mu, \lambda)$, the public key $pk_\rho = (n, g)$, and $h = g^\alpha \bmod n^2$ according to the Algorithm 2. Furthermore, the challenger C chooses the integer $\theta \in \mathbb{Z}_n^*$ and sends the private key to the adversary $\mathcal{A}$.

2) Selection: The adversary $\mathcal{A}$ selects two plaintexts $d_0, d_1 \in \mathbb{Z}_n$ and sends them to the challenger C for encryption.

3) Challenge: The challenger randomly picks $a \in \{0,1\}$ to calculate $E[[d_a]] = g^{d_a} \cdot h^\beta \cdot \theta^n \bmod n^2$, where β is a random number satisfying the $|\beta| < \frac{\rho}{2}$.

4) Guess: The adversary $\mathcal{A}$ guesses the plaintext of a in the ciphertext $E[[d_a]]$ and submits the guess $a' \in \{0,1\}$ back to the challenger C.

In this model, even if the adversary $\mathcal{A}$ possesses the private key, it cannot decrypt the ciphertext. $\mathcal{A}$ must find a random number β' that satisfies $h^{\beta'} = 1 \bmod n^2$ to get $E[[d_a]]^{\beta'} = g^{d_a \cdot \beta'} \cdot \theta^{n \cdot \beta'} \bmod n^2$ to eliminate the influence of randomness. Therefore, to obtain the correct plaintext, it is necessary to find β' that satisfies both $h^{\beta'} = 1 \bmod n^2$ and $g^{\beta'} \neq 1 \bmod n^2$. Due to the fact that Paillier encryption has $ord(g) = kn$ and $h = g^\alpha \bmod n^2$. In order to satisfy $h^{\beta'} = 1 \bmod n^2$, it is necessary to have $h^{\beta'} = g^{\alpha\beta'} = 1 \bmod n^2$; in other words, it needs to meet the following

$$\alpha \cdot \beta' = k' \cdot kn = k' \cdot k \cdot p \cdot q. \tag{19}$$

Then, we obtain

$$\beta' = \frac{k' \cdot k \cdot p \cdot q}{\alpha}, \tag{20}$$

where k' is a positive integer.

However, since $\gcd(k, \alpha) = 1$ is satisfied in the SCSA algorithm and p, q are large prime numbers, to ensure that β' is an integer, it is desired to have $k' = \zeta \cdot \alpha$, where ζ is a positive integer. Then, we have

$$\beta' = \zeta \cdot k \cdot p \cdot q = \zeta \cdot ord(g). \tag{21}$$

Therefore, observation of (21) shows that $g^{\beta'} = 1 \bmod n^2$ is established, i.e., it is not possible to satisfy $g^{\beta'} \neq 1 \bmod n^2$.

Based on this, it is clear that it is impossible to find β' that satisfies the condition. Consequently, the probability of adversary $\mathcal{A}$ guessing $a = a'$ can be hold as the following inequality

$$\Pr\left(a = a'\right) \leq \frac{1}{2} + \epsilon, \tag{22}$$

where ϵ is negligible. Thus, it can be demonstrated that the ciphertext data $E[[W_i]]$ is semantically secure under the IND-CPA. ■

Theorem 4: Given all participating entities $P = \{GS, LCs\}$. In the SCSA algorithm, for any non-uniform PPT attacker $\mathcal{A}$, a PPT simulator SIM exists to model the ideal world output as follows

$$REAL_P\left(W_i\right) \equiv SIM_P\left(W_i\right), \tag{23}$$

where $\equiv$ stands for computationally indistinguishable.

Proof: We provide a standard hybrid argument to demonstrate the security of the SCSA algorithm in the proposed DSVFL scheme. We define the simulator $SIM_P(W_i)$ by applying polynomially many hybrid modifications to the real-world output $REAL_P(W_i)$, and show that the resulting $SIM_P(W_i)$ is computationally indistinguishable from $REAL_P(W_i)$.

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.

Hyb0: Generate $REAL_P(W_i)$ after the actual execution of the SASC algorithm. Initialize multiple random variables to make them indistinguishable from $REAL_P(W_i)$.

Hyb1: We performed simulations with simulator $SIM_P(W_i)$. Each honest LC selects a random vector X_i in the appropriate distribution to replace the real plaintext data W_i, and encrypts X_i with the private key sk_{LC_i} to obtain the ciphertext $E[[X_i]]$. Theorem 3 proves that the ciphertext data satisfies IND-CPA security, so an attacker cannot distinguish between a random ciphertext and a real ciphertext.

Hyb2: In this hybrid, we require the simulator $SIM_P(W_i)$ to aggregate the random ciphertext $E[[X_i]]$ instead of the real ciphertext $E[[W_i]]$ to get the aggregation result $E[[X]]$. Given the homomorphism property of the algorithm, the result of this step is still guaranteed to be indistinguishable from the true result.

Hyb3: We define the simulator $SIM_P(W_i)$ to receive the ciphertext aggregation result $E[[X]]$ and the private key, perform the decryption, and output the decryption result X. Since SCSA decryption only involves linear operations, attacker $\mathcal{A}$ still cannot distinguish between real and simulated decryption results.

In summary, we define a PPT simulator and prove that the simulator's output $SIM_P(W_i)$ is indistinguishable from the true result $REAL_P(W_i)$, thus completing the proof. ■

Theorem 5: The SCSA algorithm ensures that GS cannot decrypt the ciphertext data of a single LC during the training process, thereby defending against GS inference attacks.

Proof: In the SCSA, the encryption and decryption processes are both carried out by LCs, making it difficult for GS to obtain the decryption private key and thus incapable of decrypting $E[[W_i]]$. Furthermore, a random $(\tau-1)$-th degree polynomial is selected using Shamir's secret sharing: $f(x) = a_0 + a_1 x + \cdots + a_{t-1}x^{\tau-1}$, where $a_0 = n$. The public key n is divided into m random numbers $n_1, n_2, \ldots, n_m$, and the random number $\theta_t \in \mathbb{Z}_n^*$ is selected as the task ID for the t-th round of training. The partial private key $\gamma_i = \theta_t^{n_i} \bmod n^2$ is computed for each LC. LC_i encrypts the plaintext as $E[[W_i]] = g^{W_i} \cdot h^{\beta_i} \cdot \theta_t^{n_i} \bmod n^2$. Consequently, even if GS obtains the decrypted private key, since n_i is only a part of the secret share of n, GS cannot deduce n and thus cannot complete the decryption. During the aggregation process, as long as τ LCs upload ciphertexts, GS can complete the aggregation and recover n. The ciphertext data is also aggregated as $E[[W]] = \prod_{i=1}^{m} E[[W_i]] \bmod n^2$. Even if GS obtains the decryption key through auxiliary information, it still cannot access the ciphertext data of LC_i. In summary, the SCSA algorithm effectively defends against the GS inference attacks. ■

C. Resist Collusion Analysis

Theorem 6: The DSVFL scheme's verification algorithm can resist GS's collusive verification with $\tau - 1$ LCs.

Proof: LC_j receives $z_j = \{z_{1j}, z_{2j}, \ldots, z_{mj}\}$ and hash value σ_e to calculate the signature. Then, LCs send their own signature to GS to aggregate as $\varpi_e = \prod_{j=1}^{\tau} Z_{ej}$. After that, GS sends the aggregated signatures to TA for verification. It follows from Theorem 2 that even if the GS colludes with 1 or $\tau - 1$ LCs it cannot achieve collusion verification by modifying the aggregation signature. This is because after GS performs aggregation on the signatures of $\tau - 1$ LCs, there is no guarantee that $\varpi_e = \sigma_e^Z$. This algorithm requires the participation of at least τ LCs to make the verification pass. However, collusion with τ LCs is very costly for the GS and is rarely realized since the τ parameter is typically set close to half the total number of LCs.■

V. Performance Evaluation

In this section, a comprehensive evaluation and analysis of the DSVFL performance is conducted. Specifically, the initial experimental setup is introduced, encompassing the simulation environment, datasets, training models, and parameter settings. Subsequently, analyses and experiments on model loss, accuracy, computational cost, communication overhead, latency are performed. The experimental results validate the effectiveness of the DSVFL.

A. Experimental Setup

1) Simulation Environment: To evaluate the DSVFL scheme, experiments are conducted using Python on a Windows 11 operating system with a 2.42 GHz Intel(R) Core(TM) i5 processor and 16 GB RAM. Prototype simulations are implemented within Python 3.8.19 and PyTorch 1.11.0.

2) Datasets: We conduct classification experiments on MNIST and CIFAR-10 datasets. The MNIST dataset, provided by the National Institute of Standards and Technology (NIST), comprises 70,000 handwritten digit images, with 60,000 used for training and 10,000 for testing. It consists of 10 categories, each representing a digit from 0 to 9, with each sample being a 28×28 grayscale image. The CIFAR-10 dataset, sourced from the Canadian Institute for Advanced Research, comprises diverse object color images. It includes 50,000 training samples and 10,000 testing samples. The dataset includes 10 categories of RGB color images, with each category containing 6,000 images, each measuring 32×32 pixels.

3) Training Models: The applicability of DSVFL is evaluated on multiple training models: a multi-layer perceptron (MLP), a convolutional neural network (LeNet), and a deep convolutional neural network (VGG-16). Specifically, the MLP network consists of 2 fully connected layers, the LeNet network includes 2 convolutional layers and 1 fully connected layer, and the VGG-16 network comprises 13 convolutional layers and 3 fully connected layers.

4) Parameter Settings: We set the number of LCs in the FL to $m = 30$. The communication rounds for models are from $T = 100$ to $T = 200$. We set the learning rate $\eta = 0.01$, batch size $\mathcal{B} = 50$, local epochs $E = 5$, and compression rate $r = 0.01$. Additionally, the security parameter of the scheme is set to $\rho = 1024$. All parameter settings are presented in the Table III.

B. Experimental Results

1) Effectiveness: We carry out several experiments to test the validity of the Ftop-k in terms of model loss and accuracy

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.

TABLE III
THE VALUE OF PARAMETER SETTINGS

Parameter	Value
number of LCs is selected, m	$[10, 30]$ (default is 30)
number of communication rounds, T	$[0, 200]$ (default is 100)
learning rate, η	$(0, 1]$ (default is 0.01)
local epoch, E	$[1, 10]$ (default is 3)
batch size, $\mathcal{B}$	$[10, 60]$ (default is 50)
compression rate, r	$[0, 1]$ (default is 0.01)
security parameter ρ	$[0, 2048]$ (default is 1024)

compared with the random-k [21] algorithm, the top-k [20] algorithm and the Gaussian$_k$ [16] algorithm. Figs. 3 and 4 illustrate the variation in model loss and accuracy with the number of iteration rounds. Since the Ftop-k selected gradients contribute more to the model training, this algorithm can exhibit the lowest model loss and higher convergence at the same compress ratio compared with other schemes. Fig. 5 illustrates the comparative performance of four compression methods. In all scenarios, Ftop-k consistently achieves lower loss and higher accuracy compared to random-k, top-k and Gaussian$_k$, demonstrating its superior effectiveness in preserving model accuracy while applying data compression. This trend is evident across various compression ratios (0.01, 0.05, and 0.1), indicating the robustness and generalizability of the Ftop-k method. These results validate the efficacy of the DSVFL scheme, highlighting its potential to enhance the performance of FL systems through efficient and reliable data compression techniques. Table IV presents the model loss and accuracy values of the random-k, top-k, Gaussian$_k$ and Ftop-k algorithms. This comparison effectively serves as an ablation study because the only difference between Ftop-k and top-k lies in the use of the error-buffer compensation. In addition, we also provide the percentage improvement in model accuracy of the Ftop-k compared to the top-k. Ftop-k achieves the highest model accuracy compared to the other three algorithms to demonstrate the effectiveness of our algorithm in improving model accuracy and reducing model loss. Furthermore, to demonstrate the superiority of the DSVFL, we compared the loss and final accuracy of the three schemes under different compression ratios and security parameters, as shown in Fig. 6. Experimental results indicate that although higher compression rates may cause some accuracy degradation, the model maintains stable convergence as training progresses. Also, increasing the safety parameter enhances safety while introducing some delay in convergence speed, but the final accuracy remains largely consistent.

2) Computational Cost: To verify the efficiency of DSVFL, we have theoretically analyzed and experimentally verified the computational cost. In addition, comparison experiments are performed with two other schemes, VPFL [22] and PPVerifier [23], with the results shown in Table V and Fig. 7.

For the sake of description, we only record and analyze complex mathematical operations. Initially, symbols are defined to prepare for the theoretical analysis. In detail, we set $\mathcal{C}_m$, $\mathcal{C}_e$, $\mathcal{C}_i$, $\mathcal{C}_h$, $\mathcal{C}_{h'}$ to denote the modular multiplication, modular exponentiation, modular inverse, the one-way hash operation and Chameleon hash operation. The number of LCs involved in training is represented by m, the gradient compression rate is r, To make the results clearer and more concise, only complex operations are considered, and simple operations are ignored in the analysis.

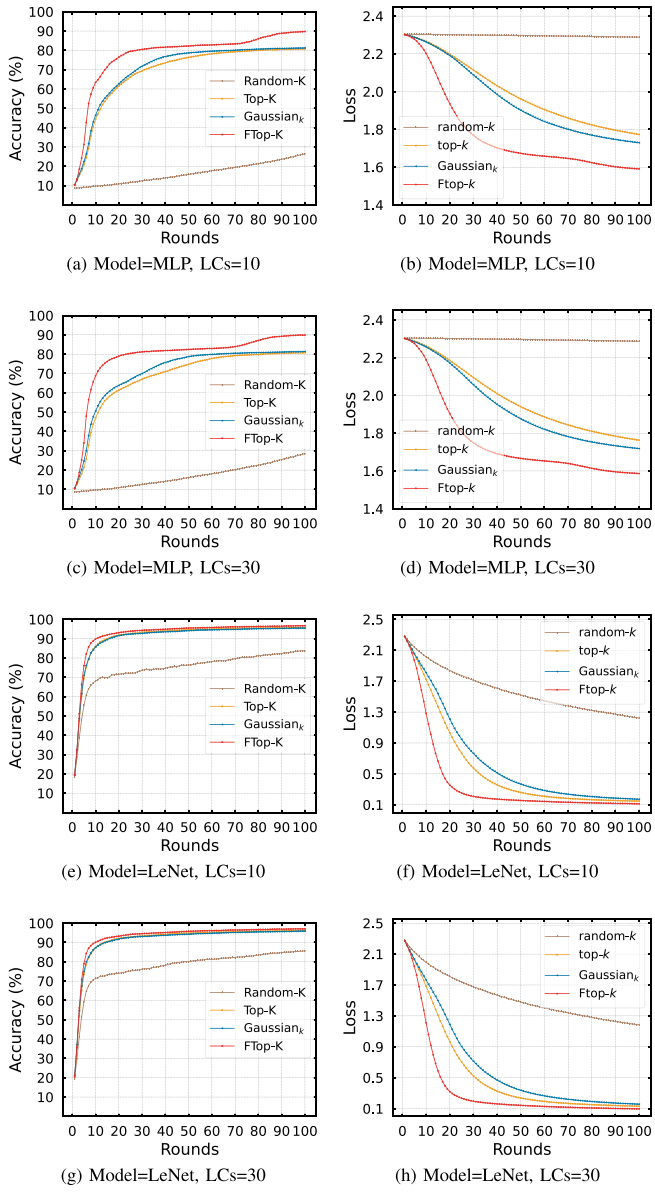


Fig. 3. The influence of the number of iteration rounds under different schemes in MNIST dataset.

In the DSVFL, LC$_i$ trains in multiple iterations, incurring the computational cost of $rd(3\mathcal{C}_e + 2\mathcal{C}_m)$. In addition, LC$_e$ computes the hash value shared with other $m-1$ LCs at the cost of $\mathcal{C}_h$. After receiving the hash values, LCs generate verification signatures using the secret shares sent by the TA, with a computational cost of $\mathcal{C}_p$. Subsequently, the GS receives the local gradients and signatures from the LCs and performs aggregation. The case where no LCs drop out is considered since, at this point, the GS incurs the highest computational cost, requiring

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
ACCURACY AND LOSS OF FOUR SCHEMES

Datasets	Model	Number	random-k [21]		top-k [20]		Gaussian$_k$ [16]		Ftop-k		Accuracy Change
			Accuracy	Loss	Accuracy	Loss	Accuracy	Loss	Accuracy	Loss	
MNIST	MLP	10	24.55%	1.9622	75.64%	1.7441	81.24%	1.7292	89.43%	1.5849	18.23% ↑
		30	28.48%	1.8791	82.82%	1.6832	71.40%	1.7185	90.69%	1.5704	9.50% ↑
	LeNet	10	22.50%	0.4575	94.48%	0.1418	95.45%	0.1720	96.83%	0.0535	2.49% ↑
		30	23.30%	0.4475	95.24%	0.148	95.91%	0.1578	97.31%	0.0608	2.17% ↑
CIFAR-10	LeNet	10	9.99%	1.9130	32.01%	1.6069	43.48%	1.6108	47.74%	1.0790	49.14% ↑
		30	10.00%	1.9163	35.13%	1.6108	45.00%	1.5765	50.62%	1.1386	44.09% ↑
	VGG-16	10	20.84%	1.8437	50.32%	1.1666	62.32%	0.8333	72.39%	0.3713	43.86% ↑
		30	31.34%	1.1580	52.62%	1.1672	65.31%	0.8488	75.87%	0.3793	44.18% ↑

TABLE V
COMPUTATIONAL COST OF THREE SCHEMES

Scheme	LC_i	GS
VPFL [22]	$(2d+3)\,\mathcal{C}_e + (d+3)\,\mathcal{C}_m + 2\mathcal{C}_h + \mathcal{C}_i + \mathcal{C}'_h$	$7\mathcal{C}_e + 2\mathcal{C}_m + 2\mathcal{C}_h + \mathcal{C}'_h$
PPVerifier [23]	$5d\mathcal{C}_e + 4d\mathcal{C}_m + 2\mathcal{C}_h$	$2\,(m-1)\,\mathcal{C}_m$
DSVFL	$(3r+1)\,d\mathcal{C}_e + (2r+1)\,d\mathcal{C}_m + \mathcal{C}_h + \mathcal{C}_p$	$\left(rd\,(m-1) + m^2\right)\mathcal{C}_m + \mathcal{C}_e + m\,(m-1)\,\mathcal{C}_i$

a total cost of $(rd(m-1)+m^2)\mathcal{C}_m + \mathcal{C}_e + m(m-1)\mathcal{C}_i$. Finally, LC_i decrypts the aggregated ciphertext with a decryption overhead of $d(\mathcal{C}_m + \mathcal{C}_e)$.

In the VPFL scheme, LC_i must be authenticated and the model parameters encrypted, with a computational cost of $(2d+3)\mathcal{C}_e + (d+3)\mathcal{C}_m + 2\mathcal{C}_h + \mathcal{C}_i + \mathcal{C}'_h$. The GS performs aggregation and decryption operations, incurring the computational cost of $7\mathcal{C}_e + 2\mathcal{C}_m + 2\mathcal{C}_h + \mathcal{C}'_h$. In the PPVerifier, LC_i requires to be blinded after the encryption of the random numbers and the gradient. At the same time, a hash operation has to be performed on gradients with the computational cost of $4d\mathcal{C}_e + 3d\mathcal{C}_m + \mathcal{C}_h$. The decryption and verification overhead is $d(\mathcal{C}_e + \mathcal{C}_m) + \mathcal{C}_h$. The computational cost of the GS aggregation is $2(m-1)\mathcal{C}_m$.

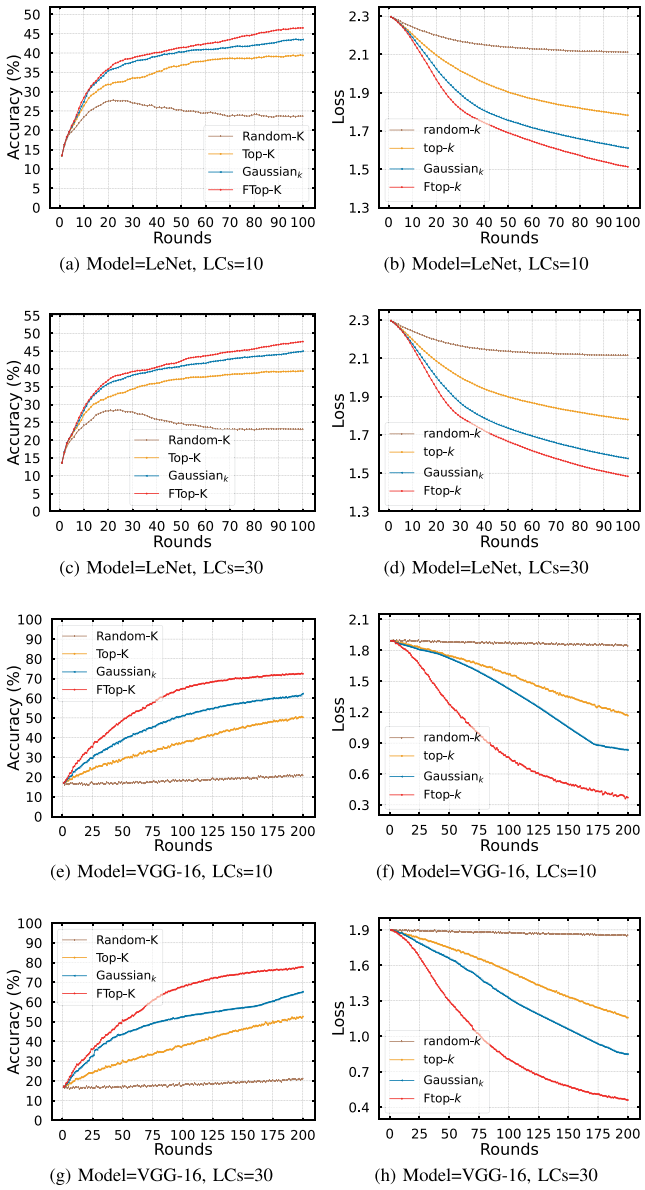
We undertook some experiments to support the theoretical analysis of communication overhead. The experimental results are shown in Fig. 7. In the simulation, we set the gradient compression rate $r = 0.01$, the number of LCs $m = 10$, the threshold $\tau = 3$, and the range of model parameter sizes N is from 10,000 to 50,000. It can be found that the computational cost of LC_i and GS in all three schemes vary according to the model parameter sizes. The increase in the number of LCs leads to an increase in the computational cost for the three schemes as the aggregation workload grows. Simultaneously, the computational cost for LC_i in the DSVFL scheme also increases as the number of signatures generated by each LC rises. In contrast, LC_i in the other two schemes remains unaffected. A comprehensive comparison reveals that the DSVFL consistently achieves lower computational cost for both LCs and the GS in scenarios with LCs=10 and LCs=30. This indicates that DSVFL is more efficient and scalable, significantly reducing the computational demands compared to VPFL and PPVerifier.

3) Communication Overhead: The communication overhead of DSVFL is compared with that of VPFL and PPVerifier. For comparison purposes, we define the one-way hash output length as b_1, the one-way hash signature length as b_2, the Chameleon hash output length as b_3 and the result of the modular exponentiation operation length is b_4. In the DSVFL scheme, LC_i uploads the encrypted compressed gradient to the GS, incurring an overhead of $2\rho rd$. Then, LC_e has to send the hash value with the other LCs, the amount of communication for the interaction is mb_1. After receiving the hash signature, LCs generate signatures based on secret sharing shares and send them to GS, with a communication overhead of b_2. GS receives the gradients and signatures from LCs and returns the aggregated results to the TA, with a total communication overhead of $b_2 + 2\rho rd$. TA verifies the aggregated signatures and ciphertext. If the verification passes, the aggregated ciphertext is transmitted to LCs for decryption, with communication overhead $2\rho mrd$.

In the VPFL, LC_i sends authentication information with local model training parameters to the GS and receives the aggregated model parameters. The total communication overhead of LC_i is $4\rho d + 2(b_1 + b_4) + b_2 + b_3$, and the total communication for GS is $md + 4b_4 + b_1$. Assuming that in the PPVerifier scheme, LC_i and GS interact cryptographic model gradients with Hash to verify signatures, the communication overhead of LC_i is $2\rho d + b_1$ and that of GS is $m(2\rho d + b_1)$.

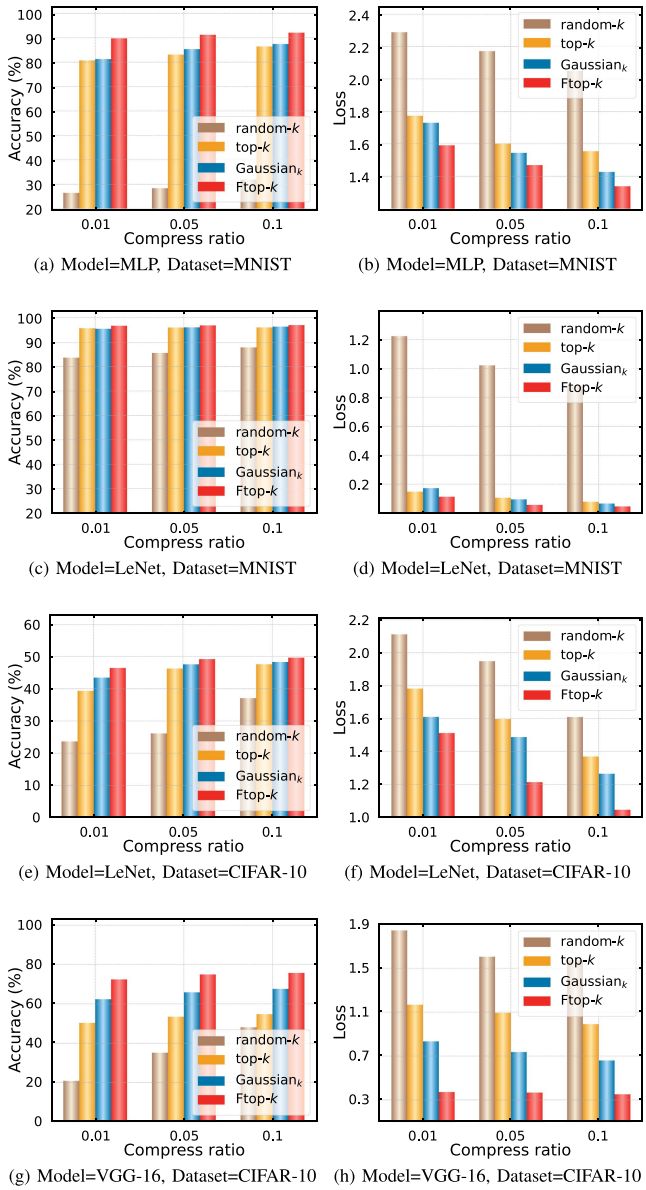
Regarding communication overhead, we have present the theoretical result in Table VI. As shown in Fig. 8, model size affects the communication overhead of LC_i and GS, with larger model sizes associated with greater communication overhead. At a model size of 10000, the communication overhead of LC_i per round in the DSVFL scheme is 19.84MB, which is much lower than the 29.71MB and 39.07MB of VPFL and PPVerifier. The communication overhead for LC_i in the DSVFL be affected by the increase in the number of LCs due to the rise in the number of signatures required for interaction. The

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.



(a) Model=LeNet, LCs=10 (b) Model=LeNet, LCs=10
(c) Model=LeNet, LCs=30 (d) Model=LeNet, LCs=30
(e) Model=VGG-16, LCs=10 (f) Model=VGG-16, LCs=10
(g) Model=VGG-16, LCs=30 (h) Model=VGG-16, LCs=30

Fig. 4. The influence of the number of iteration rounds under different schemes in CIFAR-10 dataset.



(a) Model=MLP, Dataset=MNIST (b) Model=MLP, Dataset=MNIST
(c) Model=LeNet, Dataset=MNIST (d) Model=LeNet, Dataset=MNIST
(e) Model=LeNet, Dataset=CIFAR-10 (f) Model=LeNet, Dataset=CIFAR-10
(g) Model=VGG-16, Dataset=CIFAR-10 (h) Model=VGG-16, Dataset=CIFAR-10

Fig. 5. The influence of the compression ratio under different schemes.

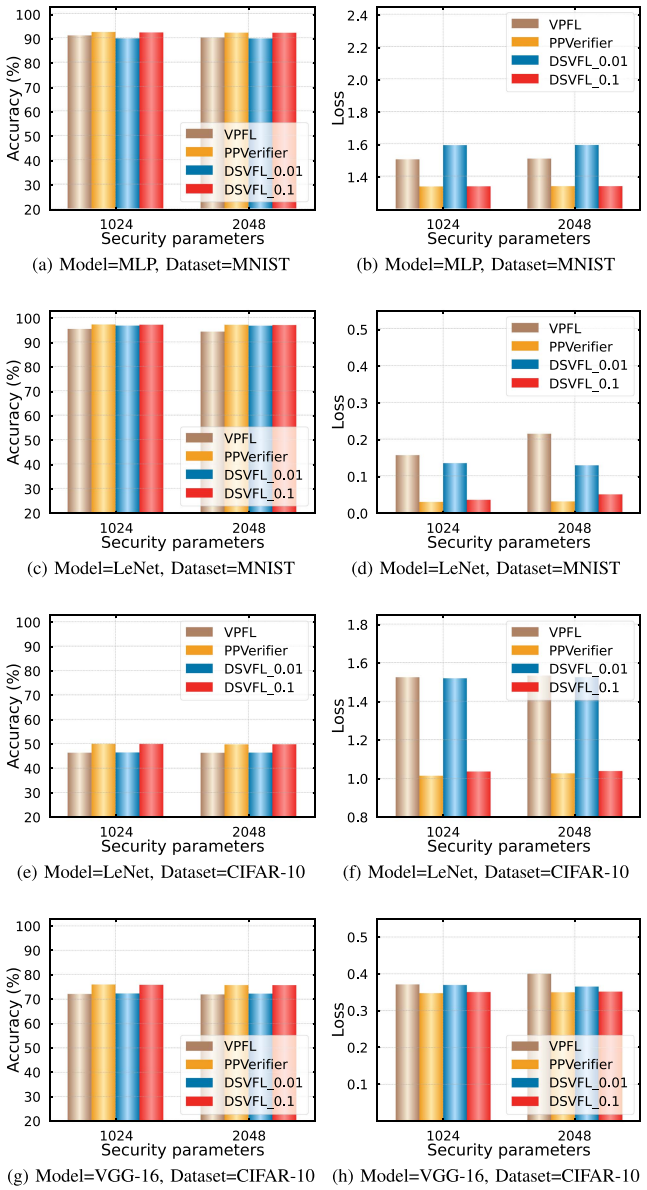
TABLE VI
COMMUNICATION OVERHEAD OF THREE SCHEMES

Scheme	LC_i	GS
VPFL [22]	$4\rho d + 2(b_1 + b_4) + b_2 + b_3$	$md + 4b_4 + b_1$
PPVerifier [23]	$2\rho d + b_1$	$m(2\rho d + b_1)$
DSVFL	$2(m+1)\rho r d + m b_1 + b_2$	$b_2 + 2\rho r d$

communication overhead of LC_i and GS are lower in the DSVFL compared to the other two schemes. These findings highlight the effectiveness of the DSVFL in optimizing communication resources in FL.

4) Latency: To verify the latency performance of DSVFL in real-world environments, we conducted end-to-end latency tests, which cover the complete process of encryption, communication, aggregation, and verification. Fig. 9 compares the total latency of the three schemes under different bandwidth with LCs=10 and LCs=30. Due to the large differences in the total latency under different models, we use logarithmic coordinates for the vertical axis in Fig. 9 to show the comparison effect more clearly. This treatment helps to present the delay gaps of different orders of magnitude without affecting the validity of the comparison. It can be observed from the comparison that the DSVFL exhibits lower overall runtimes. This result illustrates that in addition to the advantages in FLOPs and communication volume, the DSVFL also has significant advantages in operational efficiency on edge devices.

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.



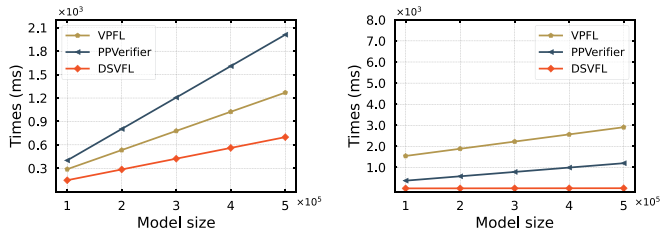
(a) Model=MLP, Dataset=MNIST (b) Model=MLP, Dataset=MNIST

(c) Model=LeNet, Dataset=MNIST (d) Model=LeNet, Dataset=MNIST

(e) Model=LeNet, Dataset=CIFAR-10 (f) Model=LeNet, Dataset=CIFAR-10

(g) Model=VGG-16, Dataset=CIFAR-10 (h) Model=VGG-16, Dataset=CIFAR-10

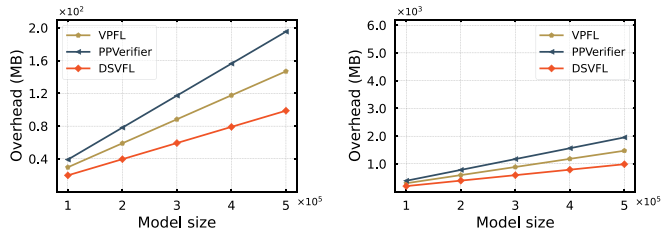
Fig. 6. The accuracy and loss of the different schemes under compression ratios and security parameters.



(a) Computational cost of the LC$_i$, LCs=10 (b) Computational cost of the GS, LCs=10

(c) Computational cost of the LC$_i$, LCs=30 (d) Computational cost of the GS, LCs=30

Fig. 7. The influence of the model size under different schemes for computational cost with LCs=10 and LCs=30.



(a) Communication overhead of the LC$_i$, LCs=10 (b) Communication overhead of the GS, LCs=10

(c) Communication overhead of the LC$_i$, LCs=30 (d) Communication overhead of the GS, LCs=30

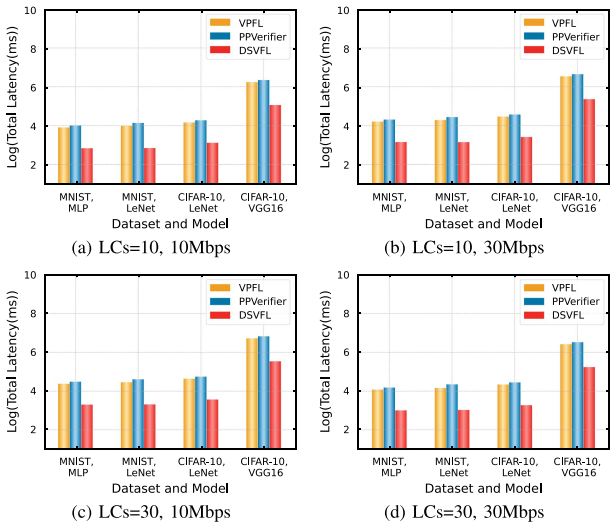
Fig. 8. The influence of the model size under different schemes for communication overhead with LCs=10 and LCs=30.

VI. Related Work

A. Gradient Compression

Research [24] indicated that 99.9% of gradient transmissions are redundant in distributed SGD algorithms. Consequently, compressed gradient techniques have been widely studied to address the communication bottleneck in distributed training. Dettmers et al. [25] reduced the 32-bit floating-point representation of the gradient to 8 bits through quantization. Bernstein et al. [26] implemented SignSGD, which updates the model by retaining only the gradient's sign, achieving 32-fold compression. However, gradient quantization impacts model performance, hindering neural network convergence. Gradient sparsity reduces the amount of communication by decreasing the number of gradients per transmission to alleviate the communication pressure. Sattler et al. [27] proposed sparse ternary compression (STC) for FL, which reduces communication volume through top-k gradient sparsification and uses an error feedback mechanism for bi-directional compression between clients and server. Wu et al. [28] indexed the top-k sparse method to enable simultaneous sharing of multiple elements. Nevertheless, all the above schemes focus solely on reducing communication overhead without addressing how to accelerated model convergence while maintaining low communication. Alistarh et al. [29] designed TopK-SGD, which drastically reduces communication overhead by transmitting only the top-k gradients in each round, and incorporates an error compensation mechanism to mitigate the information loss problem. Qian et al. [30] proposed the ECLK scheme, which incorporates an error feedback mechanism and

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.



(a) LCs=10, 10Mbps (b) LCs=10, 30Mbps

(c) LCs=30, 10Mbps (d) LCs=30, 30Mbps

Fig. 9. The latency of the three schemes under different bandwidth with LCs=10 and LCs=30.

TABLE VII
FUNCTIONALITY COMPARISON

	[22]	[23]	[32]	[35]	DSVFL
resisting collusion verification	✘	✘	✘	✘	✔
drop out tolerant	✔	✘	✘	✔	✔
model accuracy	✘	✔	✘	✔	✔
preserving global model	✘	✔	✔	✔	✔
computational efficiency	✘	✘	✔	✔	✔
communication efficiency	✘	✘	✔	✔	✔

accelerated optimization techniques to improve the communication efficiency and convergence speed by introducing a reference point and a shrinkage compressor. In contrast, the Ftop-k directly performs top-k selection on the sum of the current gradient and residuals, without introducing the learning rate. This approach preserves the important update directions and eliminates the need for auxiliary variables, making it suitable for a wide range of communication-constrained FL scenarios.

B. Privacy-Preserving in FL

FL is widely employed in model training for its distributed characterization. However, existing studies [12], [31] demonstrate that adversaries can obtain participants' private information by stealing intermediate gradients. Miao et al. [32] designed a privacy-preserving Byzantine robust FL scheme based on blockchain with full homomorphism to provide secure aggregation. Shen et al. [33] combined homomorphic encryption with support vector machine(SVM) algorithms and constructed an SVM model decision function recovery method to prevent model parameter leakage. Nevertheless, these researches only protect privacy, ignoring client dropout and verifiability issues. It was found that verifiable schemes in FL can also check the attack behavior of adversaries by verifying the correctness of the results [34]. Xu et al. [35] were the first to propose VerifyNet, a verifiable FL scheme employing homomorphic hash functions and pseudo-randomization techniques. Tang et al. [36] developed a verifiable scheme for gradients in FL, which guarantees the correctness of the gradient. However, the above schemes may fail verification due to the collaboration of any client with the server, as shown in the Table VII. Gao et al. [37] proposed a lightweight commitment scheme for irreversible gradient transformations to protect privacy, but it does not improve model performance.

VII. CONCLUSION

This paper has proposed a secure FL aggregation scheme, DSVFL, based on Paillier encryption and secret sharing for collusion resistance verification. Throughout the training process, the Ftop-k algorithm was employed for gradient sparsity, thereby reducing overhead and accelerating model convergence. Additionally, a secure aggregation protocol and the verifiable algorithm were designed to mitigate the impact of LC dropouts on the aggregation and validation processes, defending against collusion between LCs and GS. Evaluation results demonstrated that DSVFL exhibits significant advantages in model training, privacy protection, and trustworthy validation. In future work, we will further consider security defense schemes against Byzantine attacks, such as the LCs model poisoning attacks.

REFERENCES

[1] E. T. M. Beltrán et al., "Decentralized federated learning: Fundamentals, state of the art, frameworks, trends, and challenges," *IEEE Commun. Surveys Tut.*, vol. 25, no. 4, pp. 2983–3013, Fourth Quarter 2023.
[2] L. Yuan, Z. Wang, L. Sun, S. Y. Philip, and C. G. Brinton, "Decentralized federated learning: A survey and perspective," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34617–34638, Nov. 2024.
[3] R. Al-Huthaifi, T. Li, W. Huang, J. Gu, and C. Li, "Federated learning in smart cities: Privacy and security survey," *Inf. Sci.*, vol. 632, pp. 833–857, 2023.
[4] Y. Tian, S. Wang, J. Xiong, R. Bi, Z. Zhou, and M. Z. A. Bhuiyan, "Robust and privacy-preserving decentralized deep federated learning training: Focusing on digital healthcare applications," *IEEE/ACM Trans. Comput. Biol. Bioinf.*, vol. 21, no. 4, pp. 890–901, Jul./Aug. 2024.
[5] P. Chatterjee, D. Das, and D. B. Rawat, "Federated learning empowered recommendation model for financial consumer services," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 2508–2516, Feb. 2024.
[6] Q. Han, S. Yang, X. Ren, P. Zhao, C. Zhao, and Y. Wang, "PCFed: Privacy-enhanced and communication-efficient federated learning for Industrial IoTs," *IEEE Trans. Ind. Inform.*, vol. 18, no. 9, pp. 6181–6191, Sep. 2022.
[7] R. Bi et al., "Communication-efficient privacy-preserving neural network inference via arithmetic secret sharing," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 6722–6737, 2024.
[8] O. T. Sanchez, G. Borges, D. Raposo, A. Rodrigues, F. Boavida, and J. S. Silva, "Federated learning framework for LoRaWAN-enabled IIoT communication: A case study," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 24944–24957, Jul. 2025.
[9] J. Chen, J. Xue, Y. Wang, L. Huang, T. Baker, and Z. Zhou, "Privacy-preserving and traceable federated learning for data sharing in industrial IoT applications," *Expert Syst. Appl.*, vol. 213, 2023, Art. no. 119036.
[10] F. Wang, H. Zhu, R. Lu, Y. Zheng, and H. Li, "A privacy-preserving and non-interactive federated learning scheme for regression training with gradient descent," *Inf. Sci.*, vol. 552, pp. 183–200, 2021.
[11] Y. Miao et al., "RFed: Robustness-enhanced privacy-preserving federated learning against poisoning attack," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5814–5827, 2024.
[12] B. Rao, J. Zhang, D. Wu, C. Zhu, X. Sun, and B. Chen, "Privacy inference attack and defense in centralized and federated learning: A comprehensive survey," *IEEE Trans. Artif. Intell.*, vol. 6, no. 2, pp. 333–353, Feb. 2025.

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.

[13] M. Yang, H. Cheng, F. Chen, X. Liu, M. Wang, and X. Li, "Model poisoning attack in differential privacy-based federated learning," *Inf. Sci.*, vol. 630, pp. 158–172, 2023.
[14] Z. Tang, Y. Wang, and T.-H. Chang, "z-SignFedAvg: A unified stochastic sign-based compression for federated learning," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 15301–15309.
[15] A. Herzog et al., "Selective updates and adaptive masking for communication-efficient federated learning," *IEEE Trans. Green Commun. Netw.*, vol. 8, no. 2, pp. 852–864, Jun. 2024.
[16] S. Shi, X. Chu, K. C. Cheung, and S. See, "Understanding top-k sparsification in distributed deep learning," 2019, *arXiv: 1911.08772*.
[17] Y. Zhang et al., "Efficient privacy-preserving federated learning with improved compressed sensing," *IEEE Trans. Ind. Inform.*, vol. 20, no. 3, pp. 3316–3326, Mar. 2024.
[18] H. Li, Y. Sun, Y. Yu, D. Li, Z. Guan, and J. Liu, "Privacy-preserving cross-silo federated learning atop blockchain for IoT," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21176–21186, Dec. 2023.
[19] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, Springer, 1999, pp. 223–238.
[20] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descents," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2017, pp. 440–445.
[21] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1707–1718.
[22] J. Zhang, Y. Liu, D. Wu, S. Lou, B. Chen, and S. Yu, "VPFL: A verifiable privacy-preserving federated learning scheme for edge computing systems," *Digit. Commun. Netw.*, vol. 9, no. 4, pp. 981–989, 2023.
[23] L. Lin and X. Zhang, "PPVerifier: A privacy-preserving and verifiable federated learning method in cloud-edge collaborative computing environment," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8878–8892, May 2023.
[24] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
[25] T. Dettmers, "8-bit approximations for parallelism in deep learning," in *Proc. Int. Conf. Learn. Representations*, 2016, pp. 1–14.
[26] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signSGD: Compressed optimisation for non-convex problems," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 560–569.
[27] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-i.i.d data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3400–3413, Sep. 2020.
[28] D. Wu, W. Yang, H. Jin, X. Zou, W. Xia, and B. Fang, "FedComp: A federated learning compression framework for resource-constrained edge computing devices," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 43, no. 1, pp. 230–243, Jan. 2024.
[29] D. Alistarh, T. Hoefler, M. Johansson, N. Konstantinov, S. Khirirat, and C. Renggli, "The convergence of sparsified gradient methods," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 5977–5987.
[30] X. Qian, P. Richtárik, and T. Zhang, "Error compensated distributed SGD can be accelerated," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 30401–30413.
[31] O. Aouedi, K. Piamrat, G. Muller, and K. Singh, "Federated semisupervised learning for attack detection in industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 286–295, Jan. 2023.
[32] Y. Miao, Z. Liu, H. Li, K.-K. R. Choo, and R. H. Deng, "Privacy-preserving Byzantine-robust federated learning via blockchain systems," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2848–2861, 2022.
[33] G. Shen, Z. Fu, Y. Gui, W. Susilo, and M. Zhang, "Efficient and privacy-preserving online diagnosis scheme based on federated learning in E-healthcare system," *Inf. Sci.*, vol. 647, 2023, Art. no. 119261.
[34] X. Guo et al., "VeriFL: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 1736–1751, 2021.
[35] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 911–926, 2020.
[36] X. Tang, M. Shen, Q. Li, L. Zhu, T. Xue, and Q. Qu, "PILE: Robust privacy-preserving federated learning via verifiable perturbations," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5005–5023, Nov./Dec. 2023.
[37] S. Gao, J. Luo, J. Zhu, X. Dong, and W. Shi, "VCD-FL: Verifiable, collusion-resistant, and dynamic federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 3760–3773, 2023.



Mengqian Li received the BEng degree from the College of Computer and Cyber Security, Hebei Normal University, Shijiazhuang, Hebei, China, in 2019. She is currently working toward the PhD degree with the College of Computer Science and Technology, Guizhou University, Guiyang, China. Her research interests include federated learning and data security.



Youliang Tian (Senior Member, IEEE) received the PhD degree in cryptography from Xidian University, Xi'an, China, in 2012. He is a professor and the PhD supervisor with the College of Computer Science and Technology, Guizhou University, Guiyang, China. His current research interests include algorithmic game theory, cryptography and security protocols, Big Data security and privacy protection, blockchain, and electronic currency.



Junpeng Zhang (Member, IEEE) received the PhD degree from the School of Cyber Engineering, Xidian University, China, in 2025. He is currently an associate professor with the College of Computer and Cyber Security, Hebei Normal University. His research interests include privacy-preserving, security, and differential privacy.



Jinbo Xiong (Senior Member, IEEE) received the PhD degree in computer system architecture from Xidian University, Xi'an, China, in 2013. He is currently the full professor and PhD supervisor with the Industrial School of Joint Innovation, Quanzhou Vocational and Technical University, Quanzhou, China, and also with the Fujian Provincial Key Laboratory of Network Security and Cryptology, College of Computer and Cyber Security, Fujian Normal University, Fuzhou, China. His research interests include secure deep learning, data security, privacy protection, and Internet of Things.



Xinhua Cui received the ME degree in mathematics education from the College of Mathematics and Computer Science, Guizhou Normal University, Guiyang, China, in 2007. She is currently working toward the PhD degree with the College of Computer Science and Technology, Guizhou University, Guiyang. Her research interests include cryptography and information security.



Jianfeng Ma (Member, IEEE) received the PhD degree from Xidian University, Xi'an, China, in 1995. He has been a professor and the PhD supervisor with the Department of Computer Science and Technology, Xidian University since 1998. He was the special engaged professor of the Yangtze River Scholar in China. He is currently the leader of the Shaanxi Key Laboratory of Network and System Security, the Academic Committee member of State Key Laboratory of Integrated Services Networks, the council member of CCF, and an editor of the *Science China Information Sciences*, *Journal on Communications*, and *Chinese Journal of Computers*. His research interests include information and network security, wireless and mobile computing systems, and computer networks.

Authorized licensed use limited to: Anhui University. Downloaded on March 12,2026 at 16:28:21 UTC from IEEE Xplore. Restrictions apply.